

**M05 / LEARNER BOOK**

# Python for data engineering

**Standalone Zero-to-Job-Ready Data Engineer 2026**

15 controlled lessons

**A complete learning route**

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

# Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| START HERE   How to use this module . . . . .                               | 3  |
| DE05-01   CSV properly: parse, inspect, preserve raw . . . . .              | 4  |
| DE05-02   JSON and JSONL from zero . . . . .                                | 6  |
| DE05-03   Paths, folders and deterministic batch files . . . . .            | 8  |
| DE05-04   pandas profiling before transformation . . . . .                  | 10 |
| DE05-05   Cleaning with pandas and rejected rows . . . . .                  | 12 |
| DE05-06   pandas merge, cardinality and aggregation . . . . .               | 14 |
| DE05-07   Data validation: record, batch and relationship rules . . . . .   | 16 |
| DE05-08   HTTP and REST APIs from zero . . . . .                            | 18 |
| DE05-09   Calling APIs safely with requests . . . . .                       | 20 |
| DE05-10   Pagination and authentication without leaking secrets . . . . .   | 22 |
| DE05-11   Retries, rate limits and partial failure . . . . .                | 24 |
| DE05-12   Python + PostgreSQL with SQLAlchemy . . . . .                     | 26 |
| DE05-13   Configuration, environment variables and secret hygiene . . . . . | 28 |
| DE05-14   Logging, testing and reusable pipeline code . . . . .             | 30 |
| DE05-15   Integrated Python data-engineering pipeline . . . . .             | 32 |

**START HERE**

# How to use this module

Open the matching practice folder from the course root: 03\_M05/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

This book is written for a beginner. Work through the lessons in order. Read the new words, watch the assigned real instructor/source range when useful, perform the guided example on your laptop, then save an independent attempt before opening the explained review. Ready-made revision is included; you do not need to rewrite the book as notes.

Study directly from the learner book and practice files. Keep your own attempts, observed outputs and result evidence. Review material is for diagnosis after a genuine attempt, not for copying. Changed retries are separate evidence. There is no fixed time-to-job promise.

M05 turns the Python syntax from M04 into engineering behavior. You will ingest imperfect files, preserve raw evidence, separate accepted and rejected records, call a local API safely, prove pagination completeness, use bounded retries, connect Python to a database without hardcoding credentials, and finish with a repeatable mini pipeline.

The rule throughout this module is simple: a script that prints the expected-looking answer is not automatically trustworthy. You must know which input bytes were processed, what one record means, which rows were accepted or rejected, which key should remain unique, what total should reconcile, what happened on failure, and what a rerun will do.

Watch only the assigned useful source, then close it. Work the guided example from your own keyboard. Save a genuine independent attempt before opening the explained review. If review reveals a gap, close review and solve the changed retry. Preserve failed evidence instead of overwriting it.

## DE05-01

# CSV properly: parse, inspect, preserve raw

## Watch first

### [1. Corey Schafer - Python Tutorial: CSV Module - How to Read, Parse, and Write CSV Files](#)

OPTIONAL visual - no verified bounded range. Use only for CSV/DictReader context; learner book + lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Corey Schafer - Python Tutorial: CSV Module - How to Read, Parse, and [Write CSV Files](#) Link: <https://www.youtube.com/watch?v=q5uM4VKywbA> Assigned use:

OPTIONAL visual - no verified bounded range; CSV/DictReader context only. Book + lab required.

A CSV file looks simple enough to tempt beginners into treating it as ordinary text. That is dangerous. A comma can be data inside a quoted field; a newline can be represented differently across systems; an ID such as 0007 can lose meaning if something guesses it is a number. In the supplied module file, the first product is Notebook, A5. A naive `line.split(",")` would invent an extra column and silently shift the row.

Picture the raw file as evidence arriving at a receiving desk. Your first job is not to make it pretty. Your first job is to know exactly what arrived and to parse it according to the file format. Python's `csv` module understands quoting rules that a manual split does not. `csv.DictReader` also gives each parsed value a column name, which makes later validation readable.

Before transforming anything, record the source path, bytes or checksum when practical, header names and raw record count. A parser accepting the file does not prove the schema is the one your pipeline expected. `order_id,customer_id,...` and `customer,order,...` are both syntactically valid CSV headers; only one matches your contract.

Follow the source file once with `csv.DictReader`. Print only a small bounded sample, the field names and row count. Confirm that Notebook, A5 remains one product value. Then calculate a SHA-256 checksum of the source file without rewriting it. That checksum is a fingerprint of the exact bytes, not a statement that the data is correct.

A common beginner mistake is to open the raw CSV in a spreadsheet, fix a cell manually, save it, and then run the pipeline. That destroys the distinction between source and transformation. Another is to let a library infer an ID column as numeric and later wonder where leading zeros went. Raw preservation and explicit typing prevent these problems from becoming invisible.

## Follow with me once

Start by proving that a CSV parser understands a quoted comma and that the source file remains untouched.

```
from pathlib import Path
import csv, hashlib
```

```

source = Path("data/M05_Module_Orders.csv")
raw_bytes = source.read_bytes()
checksum = hashlib.sha256(raw_bytes).hexdigest()

with source.open(newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    rows = list(reader)

print(reader.fieldnames)
print("rows=", len(rows))
print("first_product=", rows[0]["product"])

print("sha256=", checksum)

```

Predict before running: there are 9 raw records and the first product must be exactly Notebook, A5, not two fields. If `split(",")` had been used, the first line would appear to have an extra value and the column alignment would be wrong.

The checksum proves which bytes you processed; it does not prove those bytes are good data. Keep those ideas separate: identity of input and quality of input are different controls.

Now inspect only bounded evidence:

```

required = {"order_id", "customer_id", "order_date", "quantity", "unit_price", "region", \
            "product"}
missing = required - set(reader.fieldnames or [])
print("missing_required_columns=", sorted(missing))
print("sample_ids=", [r["order_id"] for r in rows[:3]])

```

Expected: no required columns are missing and the first three IDs are MO01, MO02, MO03. This is a schema check, not yet a record-validity check. Later lessons decide whether each row is acceptable.

## Try it yourself

Parse `M05_Module_Orders.csv` with `csv.DictReader`. Record header, raw row count and checksum. Prove the quoted product containing a comma remains one field. Then explain why `split(",")` is not a CSV parser.

## Changed retry - only after review

Create a new CSV with a quoted city value Delhi, NCR and an ID 0009. Parse it without losing field boundaries or leading-zero identity.

## Revision in your own words

Reconstruct the core contract of CSV properly: parse, inspect, preserve raw from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-02

# JSON and JSONL from zero

## Watch first

### 1. Corey Schafer - Python Tutorial: Working with JSON Data using the json Module

OPTIONAL visual - no verified bounded range. Use only for JSON load/dump context; learner book + lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Corey Schafer - Python Tutorial: Working with JSON Data using the json  
Module Link: <https://www.youtube.com/watch?v=9N6a-VLBa2I> Assigned use:

OPTIONAL visual - no verified bounded range; JSON context only. Book + lab required.

JSON is not "a dictionary file." JSON is a serialization format that can hold objects, arrays, strings, numbers, booleans and null values. A JSON document can contain one nested object whose orders field is an array. JSON Lines (JSONL/NDJSON) instead stores one complete JSON value per line, which is convenient for streams and append-style event files. Treating the two as interchangeable leads to broken parsers and wrong record counts.

Open M05\_Nested\_Orders.json. The top level represents one batch document, not one order. The actual order grain is inside the orders array. One order has a nested customer object, one has an empty lines array, and one has customer: null. Those cases have different meanings. An empty list means the list is known and currently has no elements; null means the customer value itself is absent/unknown under this fixture.

Now inspect M05\_Order\_Events.jsonl. Each nonblank line is one event. You can process it line by line and report the line number if one record is malformed, instead of making the entire file an all-or-nothing mystery. Preserve the original line or source position in rejection evidence when possible.

When flattening nested JSON, do not casually duplicate header-level measures across child arrays and then sum them. You already saw that fan-out problem in SQL. The same grain logic applies in Python. If one order has three line objects, expanding the lines makes three rows. An order-level fee copied onto those rows is safe to display but unsafe to sum three times.

Your independent proof for this lesson should state three grains explicitly: the batch document grain, the orders array grain, and the JSONL event grain. Then change the input with one order whose customer object gains a new optional field. Decide whether that is compatible with your parser instead of silently dropping or assuming the field.

## Follow with me once

Read the nested JSON as one document, then deliberately move to the actual order array grain.

```
from pathlib import Path
import json

batch = json.loads(Path("data/M05_Nested_Orders.json").read_text(encoding="utf-8"))
print("batch_id=", batch["batch_id"])
print("orders=", len(batch["orders"]))
```

```
for order in batch["orders"]:
    customer = order.get("customer")
    customer_id = None if customer is None else customer.get("id")
    print(order["order_id"], "customer=", customer_id, "line_count=", len(order["lines"]))
```

You should see one batch document containing 3 orders. J102 has an empty lines list, while J103 has customer = None. Those are not the same condition: one says a known collection has zero members; the other says a value is absent.

Now treat JSONL differently:

```
path = Path("data/M05_Order_Events.jsonl")
events = []
with path.open(encoding="utf-8") as f:
    for line_no, line in enumerate(f, start=1):
        if not line.strip():
            continue
        event = json.loads(line)
        event["source_line"] = line_no
        events.append(event)

print("events=", len(events))
print("event_ids=", [e["event_id"] for e in events])
```

Expected: 3 events, with source lines preserved. If line 2 later becomes malformed, the rejection can say exactly which source record failed instead of only saying "the JSON file is bad."

## Try it yourself

Read the nested JSON and JSONL fixtures. State the grain of the batch, order array and event lines. Count each correctly and preserve a source locator for one missing customer case.

## Changed retry - only after review

Create a three-line JSONL file where the second event has an optional nested metadata object. Process each line and explain compatibility without forcing all events to identical shape.

## Revision in your own words

Reconstruct the core contract of JSON and JSONL from zero from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-03

# Paths, folders and deterministic batch files

## Watch first

### 1. Corey Schafer - Python Tutorial: Pathlib - The Modern Way to Handle File Paths

OPTIONAL visual - no verified bounded range. Use only for Pathlib context; learner book + lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Corey Schafer - Python Tutorial: Pathlib - The Modern Way to Handle File Paths Link: <https://www.youtube.com/watch?v=yxa-DJuuTBI> Assigned use:

OPTIONAL visual - no verified bounded range; Pathlib context only. Book + lab required.

A reliable data script must not depend on whatever folder happened to be open in the terminal. Path("data/file.csv") is resolved relative to the current working directory, which can change depending on how the script is launched. Path(\_\_file\_\_).resolve() gives you a stable anchor to the script file itself. The difference becomes important as soon as automation runs your code from a scheduler or another directory.

Picture a small landing zone with four shelves: incoming, raw, processed, and rejected. Files should move through that lifecycle by explicit rules. Do not select "the first CSV" from an unordered directory listing and hope it is today's file. Sort deterministic file lists or, better, select by a declared manifest/business date.

A deterministic batch means the same declared inputs are discovered in the same order and produce the same logical result. Determinism is not the same as idempotency: deterministic code can still append duplicates every time it reruns. We will return to safe reruns later.

Use Path.glob() or rglob() deliberately, convert the result to a sorted list if order matters, and log only safe metadata: filename, size, checksum, business date, record count. If a destination filename already exists, do not overwrite it automatically unless your contract explicitly says that the same content is being replaced. A checksum mismatch under the same source name is a data-lineage event worth surfacing.

The changed retry is deliberately different: process three files whose lexical filename order differs from their business-date order. Make the selection rule explicit and prove which order your program follows.

## Follow with me once

Anchor paths to the script, discover files deterministically, and write a manifest instead of trusting whatever directory the terminal happens to be in.

```
from pathlib import Path
import hashlib, json

HERE = Path(__file__).resolve().parent
landing = HERE / "incoming"
```

```
files = sorted(landing.glob("*.csv"))

manifest = []
for path in files:
    raw = path.read_bytes()
    manifest.append({
        "name": path.name,
        "bytes": len(raw),
        "sha256": hashlib.sha256(raw).hexdigest(),
    })

(HERE / "evidence").mkdir(exist_ok=True)
(HERE / "evidence" / "manifest.json").write_text(
    json.dumps(manifest, indent=2), encoding="utf-8"
)
```

Run the same script from two different working directories. If the script is correctly anchored, it discovers the same declared landing folder both times. That is the point of separating script location from current working directory.

A sorted file list is deterministic only with respect to the chosen sort key. If business order should be based on a date inside each file rather than filename, read that date and sort by the business key explicitly. "Sorted" is not automatically the same as "correct order."

## Try it yourself

Create incoming/raw/processed/rejected folders in your own workspace. Discover three small files deterministically and write a manifest with filename, bytes and checksum without modifying source files.

## Changed retry - only after review

Use filenames whose lexical order conflicts with a business\_date inside each file. Implement and explain the intended ordering rule.

## Revision in your own words

Reconstruct the core contract of Paths, folders and deterministic batch files from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-04

# pandas profiling before transformation

## Watch first

### [1. Luke Barousse - Python for Data Analytics](#)

4:22:56-5:08:26 - pandas intro, inspection, cleaning and analysis. Stop before copying an answer.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Luke Barousse - Python for Data Analytics Link:

<https://www.youtube.com/watch?v=wUSDVGivd-8> Assigned use: 4:22:56-5:08:26 - pandas intro, inspection, cleaning and analysis. Stop before copying an answer.

Profiling means looking before changing. In a production incident, the most expensive cleaning mistake is often a transformation applied before anyone recorded what was wrong with the source. Load the module CSV into pandas and first inspect shape, columns, dtypes, null counts, key uniqueness and a bounded sample. Do not begin with `fillna(0)` or `drop_duplicates()` merely because those commands exist.

The DataFrame has a grain just like a SQL table. Here one raw row is one delivered order record from the training file, including records that may later be rejected. The raw DataFrame therefore has nine rows. After cleaning, your accepted dataset will have a different population; comparing statistics without naming the population is misleading.

Use `df.shape`, `df.columns`, `df.dtypes`, `df.isna().sum()`, and targeted `nunique/duplicated`

checks. `describe()` is useful for numeric distributions only after you understand whether a column was parsed as numeric and whether invalid strings forced it to object/string. A column with oops in `unit_price` should make you investigate, not automatically coerce everything to zero.

Profiling outputs should be small enough to reason about. Dumping thousands of rows to the terminal is not inspection. Save a compact profile: raw row count, key duplicate count, missing counts for critical fields, parse-failure counts, and a few representative records.

In the independent task, profile the fresh inventory file without cleaning it. Predict which columns should be numeric/date/key fields, then compare that contract with what pandas inferred. Explain at least one reason inference is not business validation.

## Follow with me once

Profile before transforming. The goal is to create a compact source-state report that you can compare with later stages.

```
import pandas as pd
```

```
df = pd.read_csv("data/M05_Module_Orders.csv", dtype={"order_id": "string", "customer_id": \
    "string"})
print("shape=", df.shape)
print("columns=", list(df.columns))
```

```
print("dtypes=", df.dtypes.astype(str).to_dict())
print("nulls=", df.isna().sum().to_dict())
print("duplicate_order_ids=", int(df["order_id"].duplicated(keep=False).sum()))
```

Expected raw shape is (9, 7). The two occurrences of MO02 should be visible as a duplicate-key risk. A blank customer\_id should remain evidence of missing identity rather than being silently replaced.

Notice that pandas may infer a surprising dtype when one invalid string such as oops appears in unit\_price. That is not pandas being "wrong"; it is evidence that the source column is not

uniformly parseable as a number. Your next step is a deliberate parser/validator, not a blanket fillna(0) or forced cast that hides the bad value.

## Try it yourself

Profile M05\_Fresh\_Inventory.csv before cleaning: shape, columns, inferred dtypes, missing values, duplicate SKU count and suspected parse failures. Save the profile separately from source.

## Changed retry - only after review

Profile a changed file containing a blank numeric field and one duplicate key; do not clean it. Produce only observations and suspected risks.

## Revision in your own words

Reconstruct the core contract of pandas profiling before transformation from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-05

# Cleaning with pandas and rejected rows

## Watch first

### [1. Luke Barousse - Python for Data Analytics](#)

4:22:56-5:08:26 - pandas mechanics refresher; course rejected-row design is deeper than the video.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Luke Barousse - Python for Data Analytics Link:

<https://www.youtube.com/watch?v=wUSDVGivd-8> Assigned use: 4:22:56-5:08:26 - pandas mechanics refresher; course rejected-row design is deeper than the video.

Cleaning is a controlled transformation, not a command to "make the file work." The safest beginner pattern is to keep the raw records intact, derive parsed/cleaned values separately, and split records into accepted and rejected outputs with explicit reason codes. Rejected data is not trash; it is evidence about what the pipeline could not safely interpret or accept.

For the module practical, an order needs a nonblank customer ID, a unique order ID within the batch, a valid ISO date, a positive integer quantity and a valid nonnegative unit price. Notice how these are different kinds of checks: missing identity, batch-level uniqueness, parse validity and business-range validity. One record may fail more than one rule, so preserve all relevant reasons where practical.

Do not turn oops into zero. Zero is a valid business number with a meaning; oops is a parse failure. Do not turn a missing customer into UNKNOWN unless the downstream contract explicitly authorizes an unknown member and you preserve the provenance. These shortcuts make a pipeline "complete" by inventing facts.

The supplied validator deliberately marks both occurrences of duplicated MO02 as rejected. That policy is conservative: the batch does not tell us which of the two is authoritative. Keeping the first and deleting the second would be an assumption. Another valid system might resolve duplicates using source timestamps or version numbers, but those fields do not exist here.

Your validation contract must end with arithmetic: raw = accepted + rejected; accepted keys are unique; every rejected row has at least one reason; accepted amount can be recomputed from accepted quantity and price. Later systems will add stronger reconciliation, but this habit starts now.

## Follow with me once

Use the supplied validator as a readable example of accepted/rejected separation. Do not mutate the raw rows to make them pass.

```
import csv
from M05_Validation import validate_orders

with open("data/M05_Module_Orders.csv", newline="", encoding="utf-8") as f:
```

```
raw = list(csv.DictReader(f))

accepted, rejected = validate_orders(raw)
print("raw=", len(raw), "accepted=", len(accepted), "rejected=", len(rejected))
print("accepted_ids=", [r["order_id"] for r in accepted])
print("rejected=", [(r.get("order_id"), r["rejection_reasons"]) for r in rejected])
```

For the supplied fixture the independent controls are 9 raw = 4 accepted + 5 rejected. Accepted IDs are MO01, MO05, MO06, MO07. Both MO02 records are rejected because the batch does not tell us which duplicate is authoritative. MO09 is rejected because unit\_price='oops' is not a price; it is not converted to zero.

Then reconcile money only over accepted rows:

```
accepted_amount = sum(float(r["amount"]) for r in accepted)
assert len(raw) == len(accepted) + len(rejected)
assert len({r["order_id"] for r in accepted}) == len(accepted)
print("accepted_amount=", accepted_amount)
```

Expected accepted amount is 5100.0. A clean-looking output without these row/key/amount controls is weaker evidence than a visibly rejected bad row plus reconciliation.

## Try it yourself

Validate the module orders into accepted/rejected outputs with explicit reasons. Do not read the review controls first. Reconcile raw = accepted + rejected and prove accepted IDs unique.

## Changed retry - only after review

Use the fresh inventory fixture: valid unique SKU, positive integer quantity, nonnegative decimal cost and ISO received date. Split accepted/rejected with reasons.

## Revision in your own words

Reconstruct the core contract of Cleaning with pandas and rejected rows from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-06

# pandas merge, cardinality and aggregation

## Watch first

### [1. Luke Barousse - Python for Data Analytics](#)

OPTIONAL visual - no verified bounded range. Use only for pandas merge context; learner book owns cardinality/reconciliation.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Luke Barousse - Python for Data Analytics Link:

<https://www.youtube.com/watch?v=wUSDVGivd-8> Assigned use:

OPTIONAL visual - no verified bounded range; pandas merge context only.

A pandas merge is a join, so every SQL cardinality lesson still applies. If orders are many-to-one with customers, the customer key must be unique on the right side. `merge(..., validate="many_to_one")` can convert that assumption into an executable check instead of a comment. The `indicator=True` option can also show which rows were matched or left unmatched.

Start with the accepted order records, not the raw rejected population. Then load `M05_Customers.csv` and prove `customer_id` is unique before the merge. Record order row count and accepted amount before joining. Perform a left merge with `validate="many_to_one"`; after the merge, the order row count and accepted amount must remain unchanged. Customer attributes may be added, but one order must not become two.

Next deliberately substitute `M05_Customers_Duplicate_Key.csv`. The correct behavior is not to celebrate that pandas produced more rows. With `validate="many_to_one"`, pandas should raise a `MergeError` because the right-side key is no longer unique. That is a desirable failure: the program caught a broken relationship before money was multiplied.

Pandas has an important difference from ordinary SQL: merge documentation warns that null keys on both sides may match each other. Do not assume SQL null join semantics carry over unchanged. Critical join keys should have a deliberate missing-key policy before the merge.

For independent practice, aggregate accepted order amount by customer segment after a validated merge and prove the group totals reconcile to the pre-merge accepted amount. Then explain why a matching grand total alone would still not prove that each order was assigned to the right customer.

## Follow with me once

Turn your relationship assumption into executable code before aggregating.

```
import pandas as pd

orders = pd.DataFrame([
    {"order_id": "M001", "customer_id": "C01", "amount": 1000.0},
    {"order_id": "M005", "customer_id": "C04", "amount": 1600.0},
```

```

    {"order_id": "M006", "customer_id": "C05", "amount": 1500.0},
    {"order_id": "M007", "customer_id": "C01", "amount": 1000.0},
])
customers = pd.read_csv("data/M05_Customers.csv")

before_rows = len(orders)
before_amount = orders["amount"].sum()
merged = orders.merge(
    customers,
    on="customer_id",
    how="left",

    validate="many_to_one",
    indicator=True,
)
assert len(merged) == before_rows
assert merged["amount"].sum() == before_amount
print(merged.groupby("segment", dropna=False)["amount"].sum())

```

The merge is allowed to add customer attributes, but it is not allowed to multiply orders or money. `validate="many_to_one"` makes the right-side uniqueness contract executable.

Now substitute `M05_Customers_Duplicate_Key.csv`. The desired result is a `MergeError`, not a larger table. A controlled failure is stronger than a silent fan-out. After a valid merge, group totals should reconcile back to 5100.0. The grand total alone is not enough, however; you must also prove row preservation and key relationship because wrong customer assignments can still sum to the same grand total.

## Try it yourself

Merge accepted orders to `M05_Customers.csv` with `validate="many_to_one"` and `indicator=True`. Reconcile rows/amount before and after. Then prove the duplicate-key customer file fails the relationship contract.

## Changed retry - only after review

Create a small product dimension with one duplicate key and prove a many-to-one merge guard fails before aggregation; then repair the dimension and reconcile.

## Revision in your own words

Reconstruct the core contract of pandas merge, cardinality and aggregation from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-07

# Data validation: record, batch and relationship rules

## Watch first

### [1. Corey Schafer - Python Pydantic Tutorial: Complete Data Validation Course](#)

OPTIONAL visual - no verified bounded range. Use only for Pydantic validation context; learner book + validation lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Corey Schafer - Python Pydantic Tutorial: Complete Data Validation [Course Link: <https://www.youtube.com/watch?v=M81pfi64eeM>](#) Assigned use:

OPTIONAL visual - no verified bounded range; Pydantic context only.

Validation happens at more than one level. A record rule asks whether one row is acceptable: does quantity parse and exceed zero? A batch rule asks about the set: are order IDs unique and do accepted + rejected equal raw? A relationship rule crosses datasets: does every accepted customer\_id map to a declared customer, and is the dimension key unique? A robust pipeline names all three instead of calling every check "validation."

Pydantic is useful because it can turn many type/field rules into explicit models and error objects. But a Pydantic model is not a substitute for batch uniqueness, source-to-target reconciliation or business relationships unless you build those checks too. Type hints and successful model construction do not prove that a customer key exists in another file.

Use a small model only after you understand the manual rule. For example, parse quantity as a positive integer and unit\_price as a nonnegative decimal. Inspect the validation error rather than catching every exception and replacing the row silently. Keep the original raw record or source position next to the error evidence.

A critical distinction: technical validity and business validity differ. 2026-09-01 may parse as a date, yet still be outside the allowed business period for a particular batch. customer\_id='C999' is a valid string but may violate referential integrity. 0 is a valid integer but may violate a positive-quantity rule.

The changed retry asks you to add one syntactically valid record that fails only a relationship rule. Your explanation should identify exactly which validation layer catches it and why record-level parsing alone cannot.

## Follow with me once

Separate validation by scope. A record rule can inspect one row; a batch rule needs the whole collection; a relationship rule needs another entity or lookup.

```
from collections import Counter
```

```
rows = [  
    {"order_id": "A1", "customer_id": "C1", "qty": 2},
```

```

{"order_id": "A1", "customer_id": "C2", "qty": 1},
{"order_id": "A3", "customer_id": "C99", "qty": 0},
]
valid_customers = {"C1", "C2"}
counts = Counter(r["order_id"] for r in rows)

for row in rows:
    reasons = []
    if row["qty"] <= 0:
        reasons.append("qty_must_be_positive")           # record rule
    if counts[row["order_id"]] > 1:
        reasons.append("duplicate_order_id")             # batch rule

    if row["customer_id"] not in valid_customers:
        reasons.append("unknown_customer_id")           # relationship rule
    print(row["order_id"], reasons)

```

A1 cannot be judged duplicate by looking at one record in isolation; that fact only exists at batch scope. C99 may be syntactically valid text yet invalid against the relationship contract. Naming the scope prevents you from cramming every rule into one giant row function and then wondering why cross-record defects escape.

## Try it yourself

Classify at least six checks as record, batch or relationship rules. Implement one typed/field validation with Pydantic or a clear manual equivalent, then add one batch uniqueness and one relationship check.

## Changed retry - only after review

Add a syntactically valid customer\_id that does not exist in the dimension. Show record validation passes while relationship validation fails.

## Revision in your own words

Reconstruct the core contract of Data validation: record, batch and relationship rules from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-08

# HTTP and REST APIs from zero

## Watch first

### [1. Corey Schafer - Python Requests tutorial](#)

Optional visual - no verified bounded range. Focus on HTTP requests, response status, JSON and authentication. Practise retries and pagination with the supplied local service; this recording does not replace those controls.

### [2. DataTalksClub - Docker for Data Engineering: Postgres, Docker Compose, and Real-World Workflows](#)

Supplementary Docker/PostgreSQL workflow context. For HTTP requests, start with the Requests video above. Optional visual - no verified bounded range.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: DataTalksClub - Docker for Data Engineering: Postgres, Docker Compose, [and Real-World Workflows](#) Link: <https://www.youtube.com/watch?v=IP8xXebHmuE> Assigned use:

OPTIONAL visual - no verified bounded range; API/Python context only. Book + local simulator required.

HTTP is the transport contract behind many APIs. A client sends a request containing a method, URL, headers and sometimes a body. The server sends a response containing a status code, headers and bytes. REST is a design style layered on HTTP; you do not need to memorize philosophy before learning safe client behavior.

Picture a courier. A 200 status says the transport request succeeded according to the server; it does not prove the box contains the schema or number of records your pipeline expected. A 401 means authentication failed. A 404 means the resource was not found. A 429 means the client is being rate limited. 5xx generally means the server encountered a failure. The exact handling policy belongs to the API contract and operation.

Run M05\_Local\_API.py in one terminal. It binds only to 127.0.0.1 and provides /health, authenticated /v1/orders, /v1/flaky, /v1/rate-limit and one deliberately non-JSON endpoint. This makes failure practice repeatable even when internet access is unavailable.

For each endpoint, predict the status and content type before writing extraction logic. Do not immediately call .json() on every 200 response. The simulator's /bad-content returns status 200 with text/plain specifically to prove that transport success is not payload validity.

Independent proof: build a tiny response-contract table with method, endpoint, authentication requirement, acceptable statuses, expected media type and expected body shape. That table is more valuable than memorizing requests.get() syntax.

## Follow with me once

Treat HTTP as a request/response contract before worrying about Python syntax. Start the supplied local API in one terminal:

```
python practice/M05_Local_API.py
```

Then in another terminal inspect the health endpoint:

```
import requests

r = requests.get("http://127.0.0.1:8765/health", timeout=5)
print("status=", r.status_code)
print("content_type=", r.headers.get("Content-Type"))
print("body=", r.json())
```

Expected status is 200 and JSON body is {"status": "ok"}. Now request a missing route and observe 404. Request /v1/orders without the fictional training token and observe 401.

Those status codes are part of the contract. A server responding is not the same as a successful business response. Your code should distinguish transport failure, HTTP failure, content-type mismatch and valid JSON with unexpected fields. This mental model is what the later requests helper will enforce.

## Try it yourself

Run the local API and build a contract table for /health, /v1/orders, /bad-content, /v1/flaky and /v1/rate-limit: auth, expected statuses, media type, body shape, retry policy.

## Changed retry - only after review

Add a local endpoint/fixture expectation where status is 200 but required field is absent. Explain why transport succeeded while contract validation failed.

## Revision in your own words

Reconstruct the core contract of HTTP and REST APIs from zero from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-09

# Calling APIs safely with requests

## Watch first

### [1. Corey Schafer - Python Requests tutorial](#)

Optional visual - no verified bounded range. Focus on HTTP requests, response status, JSON and authentication. Practise retries and pagination with the supplied local service; this recording does not replace those controls.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Requests Quickstart (official) Link:

<https://requests.readthedocs.io/en/latest/user/quickstart/> Assigned use: Timeouts,

errors/exceptions, response handling. Official current reference checked 14 Sep 2026.

The Requests library makes HTTP convenient, which means it can also make unsafe code look deceptively simple. Production-style code should normally use an explicit timeout. The current Requests documentation warns that without one, calls can wait indefinitely. `raise_for_status()` converts unsuccessful HTTP statuses into exceptions, but you still need to validate media type and payload shape after transport succeeds.

Use a sequence you can explain: construct URL/params and safe headers; send with timeout; inspect status; raise for disallowed status; check expected content type; parse JSON; validate required fields. Do not log an Authorization header while debugging. If a token appears in an exception or URL, sanitize it before saving evidence.

Different failures deserve different names. A timeout is not the same as a 401; a 500 is not the same as invalid JSON; a valid JSON error object is not the same as a valid order payload. Catching Exception around the whole pipeline and printing "API failed" destroys information you need to recover.

Use the local API client to fetch one page. Then intentionally call the non-JSON endpoint and prove your media-type check fails before JSON decoding is trusted. Change the timeout to an unrealistically small value only in a controlled test if you need to see timeout behavior; do not make fragile timing tests part of mastery.

For the changed retry, call a missing endpoint. Your program should distinguish 404 from a connection failure and should not retry it as though the server were temporarily overloaded.

## Follow with me once

Use requests with three explicit safety checks: a timeout, HTTP-status handling, and response-type validation.

```
import requests

url = "http://127.0.0.1:8765/v1/orders?page=1&page_size=3"
headers = {"Authorization": "Bearer training-token"} # fictional local token only
r = requests.get(url, headers=headers, timeout=5)
r.raise_for_status()

ctype = r.headers.get("Content-Type", "").lower()
```

```
if "application/json" not in ctype:
    raise ValueError(f"Expected JSON, got {ctype!r}")

payload = r.json()
print("records=", len(payload["records"]), "next_page=", payload["next_page"])
```

With page size 3, the first page should contain 3 records and next\_page should be 2. Now call /bad-content; it returns HTTP 200 but text/plain. This proves why status 200 alone is not enough.

Never print a real Authorization header into logs or screenshots. The local training-token is deliberately fictional so you can learn the pattern without creating a real secret.

## Try it yourself

Call /health and authenticated /v1/orders with timeout + status + content-type + JSON-shape checks. Deliberately call /bad-content and a missing path and classify each failure correctly.

## Changed retry - only after review

Call a path returning 404 and prove your client does not misclassify it as JSON success or transient server failure.

## Revision in your own words

Reconstruct the core contract of Calling APIs safely with requests from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-10

# Pagination and authentication without leaking secrets

## Watch first

### [1. Using Python to consume an API with pagination - Renato Coelho](#)

04:50-13:40 - Coding API pagination; stop at Final explanation chapter boundary. Use course local API for auth/completeness evidence.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Using Python to consume an API with pagination - Renato Coelho Link: <https://www.youtube.com/watch?v=Vx6pmmRqCxl> Assigned use: 04:50-13:40 - Coding API pagination. Stop at the published Final explanation chapter boundary. Use the course local API for authentication, completeness and failure evidence.

A paginated API can return a perfectly valid first page and still give you an incomplete dataset. Pagination is therefore an integrity problem, not merely a loop exercise. The client must know how the server signals the next page/cursor and must prove the stop condition was reached.

The local `/v1/orders` endpoint returns records, page, page\_size, total and next\_page. The course client follows next\_page until it becomes null. It also tracks seen order IDs. At the end, compare fetched count, unique-key count and server-declared total. If those do not agree, do not call the extraction complete.

Authentication is configuration, not code. Set `M05_API_TOKEN=training-token` only for this fictional local simulator. In real projects, use the environment/secret mechanism approved by the platform and never commit real tokens. Your logs may say auth configured: yes; they should not print the secret value.

Pagination strategies vary: page numbers, offsets, next URLs and opaque cursors. Do not invent the next value if the contract supplies one. A cursor can encode server state and may not be mathematically incrementable. Also decide how a restart behaves: if page 3 fails after pages 1-2 were landed, can you safely replay them, or must you resume from persisted state?

Independent task: fetch all seven simulator orders with `page_size=2`, then again with `page_size=4`. The same seven unique IDs and total amount must result. Explain why changing page size should not change the logical dataset.

## Follow with me once

Pagination is a completeness problem. The first successful page is not the dataset.

```
import requests

base = "http://127.0.0.1:8765"
headers = {"Authorization": "Bearer training-token"}
rows, seen, page = [], set(), 1

while page is not None:
```

```
r = requests.get(
    f"{base}/v1/orders",
    params={"page": page, "page_size": 3},
    headers=headers,
    timeout=5,
)
r.raise_for_status()
payload = r.json()
for row in payload["records"]:
    if row["order_id"] in seen:
        raise ValueError(f"duplicate order_id {row['order_id']}")
    seen.add(row["order_id"])

    rows.append(row)
page = payload["next_page"]

print("rows=", len(rows), "unique=", len(seen), "amount=", sum(r["amount"] for r in rows))
```

The local API contains 7 unique orders totaling 7900. Repeat with page sizes 2 and 4; you must still obtain 7 unique records and the same total. If the result changes with page size, your pagination logic is incomplete or duplicating rows.

Authentication is deliberately supplied outside the URL. In real work, read the token from configuration/environment and never put it into a query string, committed file or diagnostic log.

## Try it yourself

Fetch all orders with page\_size 2 and 4. Prove fetched count == unique IDs == declared total in both cases. Keep the token out of source/logs.

## Changed retry - only after review

Fetch the same seven records with a different page size and order of page requests under a controlled replay; prove logical completeness/uniqueness remains the same.

## Revision in your own words

Reconstruct the core contract of Pagination and authentication without leaking secrets from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-11

# Retries, rate limits and partial failure

## Watch first

### [1. Corey Schafer - Python Requests tutorial](#)

Optional visual - no verified bounded range. Focus on HTTP requests, response status, JSON and authentication. Practise retries and pagination with the supplied local service; this recording does not replace those controls.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Requests Quickstart (official) Link:

<https://requests.readthedocs.io/en/latest/user/quickstart/> Assigned use: [Timeout/error](#) behavior, paired with the course fail-once/429 simulator and bounded retry policy.

Retries are controlled repetition after failures that may recover. They are not a universal cure. Connection errors, timeouts, selected 5xx responses and 429 rate limits may be transient. A deterministic 401 caused by a wrong token usually will not improve because you waited one second. Blindly retrying every status wastes resources and can amplify damage.

Bound the number of attempts. For 5xx/transient network errors, an exponential policy increases delay between attempts; jitter adds a small random component so many clients do not retry in lockstep. For 429, respect Retry-After when the server supplies it. Keep tests fast: the local simulator returns Retry-After=1 once, then succeeds.

Operation semantics matter. Retrying a GET is often safer than retrying a write whose first outcome is unknown. If a POST created a payment but the response was lost, repeating it could create two payments unless the API provides an idempotency mechanism. In M05 we focus on read/extraction requests; later pipeline modules deepen state and replay safety.

Partial failure is the more important engineering lesson. Imagine pages 1 and 2 succeeded but page 3 failed. If your program writes complete=true before page 3, the next system may consume incomplete data as though it were complete. Persist success state only after the durable output and its validation are complete.

Use /reset, then /v1/flaky: first call returns 500, the bounded client retries and the next succeeds. Repeat with /v1/rate-limit. Then remove the token and prove 401 is surfaced without a pointless retry loop. Save attempt count/status evidence, not the token.

## Follow with me once

Retry only failures that have a reasonable chance of succeeding later, and keep the retry count bounded.

The supplied client treats 429, selected 5xx responses, timeouts and connection errors as transient. It does not retry ordinary 401, 403 or 404 responses as though persistence will fix a bad credential or wrong route.

```
# after the local API is running and M05_API_TOKEN=training-token
```

```
from M05_API_Client import get_json

print(get_json("/reset"))
print(get_json("/v1/flaky", max_attempts=3))
print(get_json("/v1/rate-limit", max_attempts=3))
```

The flaky endpoint fails with 500 once, then succeeds on attempt 2. The rate-limit endpoint returns 429 + Retry-After: 1 once, then succeeds on attempt 2. The client waits according to the contract instead of hammering the service.

A retry policy must also preserve idempotency. Repeating a GET is usually safer than blindly repeating a request that creates a new record. "Retry four times" is not a universal reliability rule; it is a bounded response to a classified failure.

## Try it yourself

Reset the simulator. Prove one 500 and one 429 recover under bounded policy, while a 401 is surfaced without repeated blind retries. Save attempt/status evidence.

## Changed retry - only after review

Change retry budget from 4 to 2 and explain what evidence would show final failure. Do not create an infinite retry loop.

## Revision in your own words

Reconstruct the core contract of Retries, rate limits and partial failure from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-12

# Python + PostgreSQL with SQLAlchemy

## Watch first

### [1. Mike Bayer - SQLAlchemy 2.0 getting started \(embedded instructor video\)](#)

Optional visual - no verified bounded range. Play the first tutorial, SQLAlchemy 2.0 - The One-Point-Four-Ening. Focus on engine, connection, SELECT and transaction boundaries. The course examples use explicit transactions.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: SQLAlchemy Unified Tutorial - Engine and transactions (official) Link:

[https://docs.sqlalchemy.org/en/20/tutorial/dbapi\\_transactions.html](https://docs.sqlalchemy.org/en/20/tutorial/dbapi_transactions.html) Assigned use:

Engine/Connection, transaction scope, text(), bound parameters. Current 2.0 docs checked 14 Sep 2026.

SQLAlchemy sits between Python and database drivers. An Engine describes how to obtain connections; creating it does not necessarily establish a network connection immediately. A Connection is a scoped database resource. Transactions decide whether a group of changes commits together or rolls back when something fails.

Use your PostgreSQL environment from M03 for learner evidence when it is available. Put the URL in M05\_DATABASE\_URL; do not write username/password directly into a committed .py file. The bundled QA uses an isolated SQLite URL only to prove the SQLAlchemy code path without pretending that SQLite execution proves PostgreSQL runtime behavior. Those are different evidence levels.

The current SQLAlchemy 2.0 tutorial demonstrates two important habits: use a context manager for connection scope, and use bound parameters rather than formatting untrusted values into SQL strings. text("... WHERE amount >= :min\_amount") plus a parameter dictionary lets SQLAlchemy/driver handle the literal value safely.

For writes, with engine.begin() as conn: creates a transaction block that commits on success and rolls back if an exception escapes. That is easier to reason about than a series of untracked commits. For a pipeline load, decide which group of writes must be atomic. One giant transaction is not always right, but accidental partial commits are worse.

Independent task: create a training table under your learner schema, insert two records with bound parameters, read them back, and intentionally trigger a duplicate-key failure inside a transaction. Prove whether the failed transaction left partial new data. Save your actual PostgreSQL result if you ran PostgreSQL; label any SQLite fallback honestly.

## Follow with me once

Keep connection configuration outside code and use an explicit transaction for writes.

```
import os
from sqlalchemy import create_engine, text
```

```

url = os.environ["M05_DATABASE_URL"]
engine = create_engine(url)

with engine.begin() as conn:
    conn.execute(text("""
        CREATE TABLE IF NOT EXISTS m05_demo (
            order_id VARCHAR(20) PRIMARY KEY,
            amount NUMERIC NOT NULL
        )
    """))
    conn.execute(
        text("INSERT INTO m05_demo(order_id, amount) VALUES (:order_id, :amount)",
            [{"order_id": "DB01", "amount": 1200}, {"order_id": "DB02", "amount": 800}],
        )

with engine.connect() as conn:
    rows = conn.execute(
        text("SELECT order_id, amount FROM m05_demo WHERE amount >= :minimum"),
        {"minimum": 900},
    ).mappings().all()
    print(rows)

```

The bound parameter `:minimum` is data, not SQL text concatenation. `engine.begin()` commits the successful unit of work and rolls it back if an exception escapes the block.

For QA the package can use isolated SQLite to prove the Python/SQLAlchemy mechanics, but your learner evidence should use the PostgreSQL environment established earlier. Do not confuse a successful SQLite smoke test with proof that your PostgreSQL connection, permissions and schema context are correct.

## Try it yourself

Use your M03 PostgreSQL connection through `M05_DATABASE_URL`. Create/write/read a training table with SQLAlchemy bound parameters and one transaction-failure experiment. Label fallback evidence honestly if PostgreSQL cannot run.

## Changed retry - only after review

Insert a different two-row dataset inside one transaction, deliberately violate a unique key, and prove the transaction boundary behavior.

## Revision in your own words

Reconstruct the core contract of Python + PostgreSQL with SQLAlchemy from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-13

# Configuration, environment variables and secret hygiene

## Watch first

### [1. Luke Barousse - Python for Data Analytics](#)

5:50:56-6:27:15 - environment/project setup context; secrets policy comes from this course.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: Luke Barousse - Python for Data Analytics Link:

<https://www.youtube.com/watch?v=wUSDVGivd-8> Assigned use: 5:50:56-6:27:15 -

environment/project setup context; secrets policy comes from this course.

Configuration answers "what changes between environments without editing program logic?" A database URL, API base URL, input folder and log level are configuration. A real password/token is a secret. Keeping both outside source code reduces accidental commits and makes the same code usable on different machines.

Environment variables are one simple mechanism. Read them with `os.getenv/os.environ` and validate required values near program startup. Failing early with `M05_DATABASE_URL` is required is better than running half a pipeline and failing during the final load. Defaults are appropriate only when the default is genuinely safe and unambiguous.

A `.env` file is a convenience used by some tools/libraries; Python does not automatically load it by magic. If you use one later, keep `.env` out of version control and provide `.env.example` containing variable names and fictional placeholders, never real credentials. M06 will make `.gitignore` and repository hygiene explicit.

Do not log secrets. Also do not hide every configuration value: safe operational settings such as batch date, input path, page size and log level can be valuable run metadata. Classify configuration fields into safe-to-log versus sensitive and document the distinction.

The changed retry uses two configurations pointing to different local input folders. Without changing the program body, prove the correct folder is selected and that a missing required variable fails before any output files are written.

## Follow with me once

Configuration changes between environments; code should not need editing just because a URL or credential changes.

```
import os

api_base = os.getenv("M05_API_BASE", "http://127.0.0.1:8765")
token = os.getenv("M05_API_TOKEN")
database_url = os.getenv("M05_DATABASE_URL")

if not token:
    raise RuntimeError("M05_API_TOKEN is required")
```

```
if not database_url:
    raise RuntimeError("M05_DATABASE_URL is required")

print("api_base=", api_base)
print("token_present=", bool(token))
print("database_configured=", bool(database_url))
```

Notice what is not printed: the token itself and the full credential-bearing database URL. Logs should prove configuration presence without exposing secrets.

A .env file can be convenient locally, but it is still a secret-bearing file and must be ignored by Git. .gitignore prevents future accidental tracking; it does not erase a secret that was already committed. M06 will teach the repository side of that boundary.

## Try it yourself

Create a safe configuration check that requires M05\_API\_BASE and M05\_API\_TOKEN at startup. Produce .env.example with placeholders only, and a note classifying which settings may be logged.

## Changed retry - only after review

Switch the input directory via environment variable without editing program logic; then remove one required variable and prove failure occurs before outputs.

## Revision in your own words

Reconstruct the core contract of Configuration, environment variables and secret hygiene from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-14

# Logging, testing and reusable pipeline code

## Watch first

### [1. PyCon US 2025 - Building Resilient Data Pipelines: The Power of Idempotency](#)

OPTIONAL visual - no verified bounded range. Use only for idempotency/testing context; learner book + tests are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: PyCon US 2025 - Building Resilient Data Pipelines: The Power of [Idempotency](#)  
Link: <https://www.youtube.com/watch?v=kT8YWX4NE2o> Assigned use:

OPTIONAL visual - no verified bounded range; idempotency/testing context only.

A pipeline needs observable behavior and tests that protect its contracts. Logging is structured evidence about what the program did: run ID, source file, counts, durations, status and error class. Logging started and finished alone does not prove correctness. Never include tokens/passwords or unnecessarily dump personal/raw data into logs.

Use levels deliberately: DEBUG for detailed development diagnostics, INFO for normal lifecycle milestones, WARNING for recoverable abnormal conditions, ERROR for failed operations. A useful error message identifies the operation and safe context, while the exception/traceback preserves technical detail. Do not catch an error only to replace it with a vague string.

Unit tests isolate small deterministic functions such as `validate_orders`. Integration tests check boundaries between parts: reading a fixture, calling the local API, or writing/reading a database. A test should assert an invariant, not merely that no exception occurred. The visible tests prove a valid record computes amount correctly and a duplicated key is rejected.

Idempotency means repeating the same logical operation does not multiply the intended state. The PyCon 2025 source is useful because it frames idempotency as a pipeline design criterion and connects it to testing. In this module, your file-output checkpoint should overwrite or atomically replace a declared run output rather than append accepted rows again on every rerun. Later modules add more sophisticated checkpoints and incremental state.

Independent task: add one test for an invalid date and one for missing customer ID. Then run the same reference pipeline twice and compare row counts/checksum or logical outputs. If the second run doubles rows, the design is not safe to rerun.

## Follow with me once

Make the pipeline observable and testable without turning every function into a print statement.

```
import logging

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
log = logging.getLogger("m05")
```

```
def reconcile(raw_count, accepted_count, rejected_count):
    if raw_count != accepted_count + rejected_count:
        raise AssertionError("row reconciliation failed")
    log.info(
        "reconciled raw=%s accepted=%s rejected=%s",
        raw_count, accepted_count, rejected_count,
    )

reconcile(9, 4, 5)
```

A useful log states what happened without dumping confidential records or tokens. A useful test asserts a behavior such as duplicate IDs being rejected. The supplied

M05\_Visible\_Tests.py contains one valid-record test and one duplicate-ID test; run it before changing the validator, then add an edge case of your own.

Idempotency is a separate property: if you rerun the same valid input, the logical result should not silently double. A deterministic overwrite of derived files can be idempotent; blind append usually is not. Later orchestration modules will depend heavily on this habit.

## Try it yourself

Add two visible unit tests to the validation module and run them. Then run an identical source twice and prove the output population does not double. Capture a safe INFO log with counts but no secrets.

## Changed retry - only after review

Write a test for invalid date and one for duplicate ID policy. Break one rule intentionally, observe failure, repair only that rule and rerun.

## Revision in your own words

Reconstruct the core contract of Logging, testing and reusable pipeline code from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.

## DE05-15

# Integrated Python data-engineering pipeline

## Watch first

[1. DataTalksClub - Docker for Data Engineering: Postgres, Docker Compose, and Real-World Workflows](#)

OPTIONAL visual - no verified bounded range. Use only for end-to-end workflow context; integrated course checkpoint is required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Instructor / source: DataTalksClub - Docker for Data Engineering: Postgres, Docker Compose, [and Real-World Workflows](#) Link: <https://www.youtube.com/watch?v=IP8xXebHmuE> Assigned use:

OPTIONAL visual - no verified bounded range; workflow context only.

The integrated pipeline combines the habits of the module into one defensible flow: identify exact source -> preserve raw identity/checksum -> parse -> validate -> split accepted/rejected -> transform accepted records -> optionally enrich/load -> reconcile -> save run metadata -> prove rerun behavior. Each arrow is a contract boundary, not merely a function call.

Your module practical uses nine raw orders. Do not begin by reading the expected answer from the review. Build from M05\_Checkpoint\_TODO.py in your own working folder. Save the first attempt even if it fails. A good solution will surface the invalid records with reasons rather than editing them away; preserve source row numbers or equivalent provenance so a reviewer can trace each rejection.

The core reconciliation is multidimensional. Raw row count must equal accepted plus rejected. Accepted order IDs must be unique. Every rejected row must have a reason. Accepted amount must equal the independent sum of accepted quantity x price. Source checksum/run metadata must point to the exact input. If you enrich with customers, merge cardinality must not change accepted order grain or amount.

Then deliberately create a failure. Use the duplicate-customer file with a many-to-one merge assertion, or replace the expected order schema with a malformed file. The pipeline should stop at the first violated contract and should not publish a misleading "successful" final artifact. Keep the failed evidence before repairing it.

Finally rerun the same exact source. The logical output should not grow simply because the command was executed twice. After review, close the answer book and solve the fresh inventory checkpoint with different columns, identifiers, validation rules and total. That fresh transfer is the real mastery evidence for M05.

## Follow with me once

Run the complete reference flow once only after you understand the earlier pieces. It reads the exact source, validates it, separates accepted/rejected records and writes run metadata.

```
python practice/M05_Pipeline_Reference.py
```

The supplied module fixture should report:

```
raw_rows = 9
accepted_rows = 4
rejected_rows = 5
accepted_amount = 5100.0
```

Now run it a second time with the same source. The logical controls should remain the same; the output files should be replaced deterministically rather than accumulating duplicate rows.

Do not stop at "the script ran." Open the generated metadata and independently verify: source filename/checksum, raw = accepted + rejected, accepted IDs unique, accepted amount recomputes, every rejected record has a reason, and the original source file checksum did not change.

Finally run the fresh inventory checkpoint only after review is closed. Its entity/key/rules differ from the order fixture, so passing it is stronger evidence than rerunning memorized order code.

## Try it yourself

Complete M05\_Checkpoint\_TODO.py independently. Preserve first attempt, exact source identity, accepted/rejected outputs, reconciliation, run metadata, one deliberate failure and rerun evidence.

## Changed retry - only after review

After reviewing your weak points, close all review material and solve M05\_Fresh\_Checkpoint\_TODO.py from the fresh inventory fixture. Keep it separate from the first attempt.

## Revision in your own words

Reconstruct the core contract of Integrated Python data-engineering pipeline from memory. State the input grain/state, one failure mode, one independent validation, and what evidence you would keep. Then do one fresh 10-20 minute micro-task without video, review or AI solving open.

## Evidence to keep

Save code/config used, observed output or error, one independent validation, one own-words explanation, and any repair/retry as a separate artifact. Do not overwrite the first attempt.